



L-Net: A lightweight convolutional neural network for devices with low computing power

Hua Shen, Zhiwei Wang, Jixin Zhang^{*}, Mingwu Zhang^{*}

School of Computer Science, Hubei University of Technology, China

ARTICLE INFO

Keywords:

Deep learning
Depthwise separable convolution
Residual learning
Attention mechanism
Object recognition

ABSTRACT

Deep learning (DL) has demonstrated exceptional success across various domains, including computer vision, natural language processing, and speech recognition. However, the training and inference processes of DL models typically require substantial computational resources and storage space, presenting a significant challenge within the Internet of Things (IoT) domain. This study contributes theoretically to the field of lightweight DL by proposing L-Net, a lightweight convolutional neural network designed specifically for low-compute devices. The L-Net addresses challenges associated with channel interaction disparities and vanishing gradients. To further improve the network performance, we introduce the residual enhanced channel attention (or RECA) module, which combines a bypass mechanism derived from simplified residual learning with the attention mechanism's cross-channel interaction. Additionally, we replace the rectified linear unit function (or ReLU) with an exponential linear unit (or ELU) function to enhance the network's nonlinear expression capability and training speed. We conducted object recognition experiments and compared the accuracy and prediction stability of L-Net with well-known models, such as AlexNet, VGG11, SqueezeNet, ResNet, and MobileNet, to assess its efficacy. Using the CIFAR-10 dataset and our custom dataset of apple tree leaf diseases, our experimental results demonstrate that, with relatively smaller model parameters, L-Net performs exceptionally well in terms of mean Average Precision (*mAP*), achieving 0.906. Furthermore, when applied to our custom dataset, L-Net exhibits relatively consistent performance across various dataset splits under different ratios, outperforming the majority of models.

1. Introduction

Deep learning (DL) has brought about a revolution in various domains, including computer vision, natural language processing, and speech recognition. However, the widespread use of deep neural networks has posed significant challenges, primarily due to their high computational complexity and substantial memory requirements. These challenges are particularly pronounced in scenarios that involve limited resources, such as the Internet of Things (IoT) domain. The critical and urgent issue lies in deploying resource-intensive DL models on IoT devices with constrained compute capabilities, power sources, and memory capacities. In response, researchers have focused on developing lightweight DL networks that strike a balance between model size and performance. These networks are specifically designed for computational efficiency, with reduced parameter counts and optimized architectures, making them suitable for effective deployment on IoT devices with limited resources. In 2015, He et al. [11] introduced ResNet, which

^{*} Corresponding author.

E-mail addresses: nancy78733@126.com (H. Shen), 1366768167@qq.com (Z. Wang), zhangjx@hbut.edu.cn (J. Zhang), csmwzhang@gmail.com (M. Zhang).

revolutionized the DL field by addressing the problem of vanishing gradients through residual connections. Another significant milestone in lightweight networks was the development of SqueezeNet by Iandola et al. [16], which achieved substantial model compression while maintaining competitive accuracy. MobileNet, proposed by Howard et al. [12], further advanced the field by introducing depthwise separable convolutions, which greatly reduced computational costs while preserving the ability to capture complex features. This approach laid the foundation for efficient DL models on mobile and edge devices.

However, ResNet was not originally designed as a lightweight network and had a large number of parameters, rendering it impractical for devices with limited computational power. Similarly, SqueezeNet encountered challenges in terms of reduced accuracy, prolonged training times, and limited channel interaction. Likewise, MobileNet also has its limitations, including poor channel interaction and lower accuracy.

Despite advancements in lightweight deep learning (DL) networks, challenges still persist, particularly related to issues such as channel interaction disparity and gradient vanishing. This study introduces L-Net, a lightweight convolutional neural network (CNN) that addresses these challenges by incorporating the residual-enhanced channel attention (R-ECA) module into the depthwise separable CNN. L-Net integrates a bypass mechanism that simplifies residual learning and combines it with the attention mechanism's cross-channel interaction capabilities. This integration improves CNN performance and mitigates the limitations of fixed residual module structures. Additionally, the exponential linear unit (ELU) activation function is utilized instead of the rectified linear unit (ReLU). This substitution enhances cross-channel interaction and accelerates training. The R-ECA module serves as a plug-and-play solution, effectively resolving issues such as information loss, inadequate cross-channel interaction, and gradient vanishing in a unified manner.

In summary, This study aims to develop lightweight deep learning networks that are specifically designed for IoT devices with computational limitations. The innovative R-ECA module introduced in this study effectively tackles challenges associated with channel interaction disparity and gradient vanishing. Consequently, it leads to the development of a high-performance and efficient convolutional neural network (CNN). The experimental results substantiate the superiority of L-Net in comparison to existing models. This highlights the potential of L-Net for real-world applications in environments with resource constraints. The following sections of the paper present a detailed analysis of related works, the construction process of L-Net, the experimental results, and concluding remarks.

2. Related work

2.1. The development process of DL

DL, a fundamental technique for realizing artificial intelligence, has witnessed rapid development. It began with LeNet [19], an early DL method primarily utilized for recognizing and classifying handwritten characters. LeNet laid the foundation for CNNs, which form the basis for subsequent DL networks. AlexNet [18] was the first to demonstrate the superiority of DL features over manually designed features, leading to its victory at the ImageNet Large Scale Visual Recognition Challenge 2012 [5]. In 2014, Girshick et al. introduced the region based-CNN (R-CNN) [9], which significantly improved performance on the PASCAL dataset [6] by almost 50% compared to prior methods. However, R-CNNs suffer from image distortion issues caused by scaling. To address this issue, He et al. [10] proposed SPP-Net, which introduced a spatial pyramid pooling (SPP) layer to extract features from different-scale regions without scaling the data region. Subsequently, Girshick et al. [8] proposed Fast R-CNN, which used the entire image and candidate regions as inputs, replacing the SPP layer with the region of interest (RoI) layer. Fast R-CNNs employed Softmax for secondary classification, employed multitask training, and achieved improved performance. In 2015, Ren et al. [28] introduced Faster R-CNN, an innovative object detection algorithm that incorporated a region proposal network (RPN) instead of a selective search algorithm. The RPN, with its multi-reference window mechanism, enabled Faster R-CNN to rapidly generate candidate region proposals within one model, making it the first object-detection algorithm to conduct end-to-end training and real-time performance on a GPU. Around the same time, Redmon [27] proposed YOLO (You Only Look Once), a pioneering single-stage DL detector widely recognized for its exceptional processing speed. Liu et al. [24] later combined the speed of YOLO with the accuracy of Fast R-CNN to develop the single shot detector (or SSD) algorithm. In 2019, the Google Brain team [7] introduced *theneuralarchitecturesearchfeaturepyramidnetwork(NAS-FPN)*, a feature pyramid structure discovered using a neural network structure search. It integrates features at different scales through top-down and bottom-up connections. The architecture of NAS-FPN expands during the search process through iterative FPN connections. To address resource constraints in various scenarios, the Google Brain team introduced EfficientDet [33] in November 2019. Compared to traditional object detection methods, the EfficientDet series significantly reduced the number of parameters (by 4 – 9 times) and the number of floating point operations per second (FLOPs) (by 12 – 42 times). Remarkably, EfficientDet-D7 achieved a remarkable 55.1% average precision (AP) on the MS-COCO dataset with a single model and scale.

In recent years, there has been a growing trend towards the use of neural networks in devices such as smartphones, wearables, and IoT devices. However, these devices frequently encounter constraints in terms of computing power, storage capacity, and energy, which pose challenges when deploying conventional DL models. One crucial challenge in designing neural networks for low-power devices involves balancing accuracy with a reduced number of parameters and computations. DenseNet and MobileNet are two notable network architectures developed to address this challenge. DenseNet [13] aims to enhance the flow of information between layers and mitigate the issue of vanishing gradients, a common problem in complex networks. By enabling each layer to receive input from all previous layers, DenseNet ensures smooth propagation of the gradient signal across the entire network. In contrast, MobileNet [12] addresses this challenge by reducing the computational requirements. It replaces standard convolutions with

depthwise separable convolutions that break the convolutional operation into depthwise and pointwise convolutions. This approach significantly reduces the number of parameters and computations required, while maintaining high accuracy. However, MobileNet has its limitations. For example, the use of the ReLU activation function after depthwise separable convolution can lead to information loss, resulting in reduced accuracy and stability. Additionally, the fast-pass interaction of depthwise separable convolution can compromise accuracy. The size of objects can vary significantly, affecting the accuracy of object detection. To address this issue, Zhou et al. [43] presented scale normalization for the image pyramids (SNIP) method. SNIP optimizes the utilization of all training data by training each scale of the image pyramid, ultimately improving classification accuracy. Dai et al. [4] proposed another approach that introduces a novel feature-level super-resolution method. This method employs dilated convolutions to ensure that high-resolution target features have the same receptive field size as low-resolution features generated by the feature extractor, effectively preventing the generation of erroneous super-resolution features. Zhang et al. [41] also presented an adaptive resampling technique for data augmentation that migrates the background and scale-mismatch issues when pasting small objects. PeleeNet [35] and ShuffleNetV1 [42] are alternative architectures developed to optimize computation and enhance accuracy. PeleeNet uses a novel architecture based on traditional convolutions and computational conservation techniques. By contrast, ShuffleNetV2 adopts a building block that splits the input into two parts and connects them using depthwise separable convolutional layers. Xiong et al. [37] introduced MobileDet, a mobile-friendly object detection framework based on lightweight CNNs. MobileDet features a dynamic feature pyramid (or DFP) to adjust the feature layer depth and width dynamically for different object scales. Additionally, it utilizes a lightweight detection head (or LDT) with depthwise separable convolutions and attention mechanisms to reduce parameter count and computational load, enabling efficient object detection on mobile devices. Ye et al. [39] presented LightSeg as a lightweight semantic segmentation network. LightSeg employs a novel encoder-decoder structure called the lightweight encoder-decoder (or LED) that utilizes lightweight convolutions and skip connections for efficient feature extraction and reconstruction. A lightweight upsampling module (or LUM) is used with transpose convolutions and residual connections to achieve high-resolution semantic segmentation in real-time applications. Huang et al. [14] proposed UNet3+, a full-scale connected UNet for medical image segmentation. UNet3+ leverages comprehensive skip connections and deep supervision to aggregate low-level details and high-level semantics from feature maps at different scales. This approach is particularly advantageous for organs of various scales. In addition to improving accuracy, UNet3+ reduces the network parameters to enhance computational efficiency. It also introduces a hybrid loss function and a classification-guided module to enhance organ boundary delineation and produce more accurate segmentation results, which are validated on two datasets. However, as the depth of these networks increases, concerns regarding gradient disappearance and explosion can arise, significantly affecting both training and accuracy. The primary goal of this study is to address these issues.

Deep learning techniques have been widely applied in various fields including remote sensing image processing, facial recognition, and agriculture. Li et al. [21] improved the accuracy of remote sensing image object detection by enhancing the YOLOv5 algorithm, providing technical support for the processing of remote sensing images. Lin et al. [22] optimized the Mask-RCNN algorithm to extract high-precision remote sensing images effectively. Additionally, Chen et al. [3] improved the multi-scale feature fusion SSD method, with a focus on detecting small targets in remote sensing images and successfully reducing the miss detection rate. Sheng et al. [2] proposed the lightweight MobileFaceNets model designed for real-time face verification tasks on mobile devices with high accuracy. Additionally, Zhang et al. [40] have employed a U-PSP-Net convolutional neural network structure to verify the feasibility and effectiveness of neural network structures in glacier remote sensing images with shadows. In the field of agricultural image processing, Liu et al. [23] have proposed an iterative pruning VGGNet method, which outperforms lightweight backbone networks in terms of performance. Huang et al. [15] have improved the Xception network model and introduced a more accurate image classification method. However, there might be overfitting issues on small datasets, especially when applying improved lightweight convolutional networks to grape diseases. Ma et al. [25] have conducted a research study on deep learning-based classification and recognition of tomato leaf diseases. Early diagnosis and treatment of tomato diseases are beneficial for enhancing tomato yield. Furthermore, Wang et al. [36] have proposed the MCA-MobileNet model, which exhibits high computational efficiency and a smaller model size, making it suitable for resource-constrained scenarios such as mobile devices. Li et al. [20] aimed to accurately and quickly identify crop diseases, reduce the cost of artificial diagnosis, and minimize the impacts of crop diseases on crop yield and quality, they proposed an improved YOLOX-Nano intelligent detection and recognition model based on a convolution attention mechanism. Yang et al. [38], on the other hand, focused on the high computational complexity and slow processing speed of the YOLOv5 model in rice disease leaf detection. They proposed a rice disease leaf recognition and detection method based on an improved SPP-x YOLOv5 model. Overall, these research achievements encompass a wide range of applications of deep learning in remote sensing images, facial recognition, and agriculture, providing rich experiences and insights for the development of relevant fields. However, there is currently limited research in the domain of apple leaf diseases. This study applies the L-Net model to the field of apple disease and demonstrates its effectiveness to a certain extent.

2.2. The work of competitors

ResNet [11] has contributed significantly to the field of DL by introducing residual connections. These connections effectively mitigate the issue of gradient vanishing, thereby enabling the training of deeper networks and facilitating successful optimization of models with numerous layers. This advancement has expanded the possibilities of constructing models that are both more powerful and expressive.

SqueezeNet [16] is another notable contribution in DL that emphasizes model compression to reduce the size of deep neural networks without compromising their performance. SqueezeNet achieves impressive compression ratios while maintaining competitive

Table 1
Interpretation of notation.

Notation description	Notation interpretation
C_{in}	number of ordinary convolution input channels
C_{out}	number of ordinary convolution output channels
D_H, D_W	size of feature map
D_K	convolutional kernel size
D_F	size of Depthwise feature maps
W_i	1×1 convolution operation
W_{dp}	depthwise separable convolution operation
$H()$	dimensionality augmentation
$F()$	a series of depthwise separable convolution
$D = \{(x^{(n)}, y^{(n)})\}_{n=1}^N$	train sets
L	the number of layers in the network
J	loss rate
σ	Backpropagation computes the error

accuracy by utilizing 1×1 convolutions and squeeze excitation modules. It is particular useful in resource-constrained environments where storage and computational resources are limited.

MobileNet [12] presents an innovative approach to decrease the computational cost of DL models. By employing depthwise separable convolutions, MobileNet substantially reduces the number of parameters and computations required, while still capturing intricate features. This methodology facilitates faster inference times and more efficient deployment of DL models on mobile and embedded devices.

However, ResNet is not suitable for devices with low computing power due to its large number of parameters. Although SqueezeNet is lightweight, it has limitations in terms of accuracy, training time, and channel interaction consistency. Conversely, MobileNet suffers from poor channel interaction. The proposed L-Net addresses these problems and incorporates a bypass mechanism to provide additional flexibility and scalability in the network architecture.

3. Preliminary knowledge

For ease of reading, Table 1 presents the main notations and their description.

3.1. Working principle of depthwise separable convolution

The principle of depthwise separable convolution involves dividing a standard convolution layer into two distinct parts: Depthwise and Pointwise. Each input channel has its own independent convolution core that collects the characteristics of the channel. The number of input and output channels remains the same; thus, the Depthwise part does not alter the dimension. The Pointwise part functions using a 1×1 convolution and collects the characteristics of each point, potentially increasing or decreasing the dimension. The working characteristics of ordinary convolution and a depthwise separable convolution are illustrated in Fig. 1. In 2017, Google introduced the lightweight model MobileNetV1, whose backbone network is built using a depthwise separable convolution.

3.2. The limitation of depthwise separable convolution

Although it is possible for $N \times N$ Depthwise and 1×1 Pointwise to achieve performance similar to that of $N \times N$ standard convolution in theory, the small dimensions of each Depthwise kernel can cause sparsity in the output of the convolution owing to the influence of the ReLU activation function. This limitation may become more pronounced during fixed-point and low-accuracy training. Additionally, because the data only exists within each channel, there is no interaction between channels, which can result in information shielding, blocking, and no circulation. Moreover, with an increase in the number of depthwise separable convolution layers, the model can suffer from gradient vanishing problems.

3.3. The parameter comparison between depthwise separable convolution and standard convolution

As illustrated in Fig. 1, the operation amount of a standard convolution is $FLOP_s = (2 \times C_{in} \times D_K^2 - 1) \times D_H \times D_W \times C_{out}$ and the parameter amount is $D_K^2 \times C_{in} \times C_{out}$. Here C_{in} and C_{out} are the numbers of input and output channels, respectively, D_K is the size of a convolution kernel; D_H and D_W are the sizes of an input feature map. The depthwise separable convolution comprises two phases: Depthwise and Pointwise. Supposing that the input feature dimensions are $D_H \times D_W \times C_{in}$, then the number of the parameters of the convolution kernel was $D_K \times D_K \times 1 \times C_{in}$, the output feature dimension was $D_H \times D_W \times C_{in}$. During the convolution process of the Depthwise phase, each channel only corresponds to one convolution core (that is, scan depth is 1). Thus, the number of operation in this phase is $FLOP_s = C_{in} \times D_H \times D_W \times D_K^2$. The input of the Pointwise phase was the output feature of the Depthwise phase; therefore, the input dimensions were $D_H \times D_W \times C_{in}$. The number of the parameters of the convolution kernel was $1 \times 1 \times C_{in} \times C_{out}$. During the convolution process, a 1×1 standard convolution was performed for each feature. Therefore, the number of operations in this phase was $FLOP_s = C_{out} \times D_F^2 \times C_{in}$. From above, we have

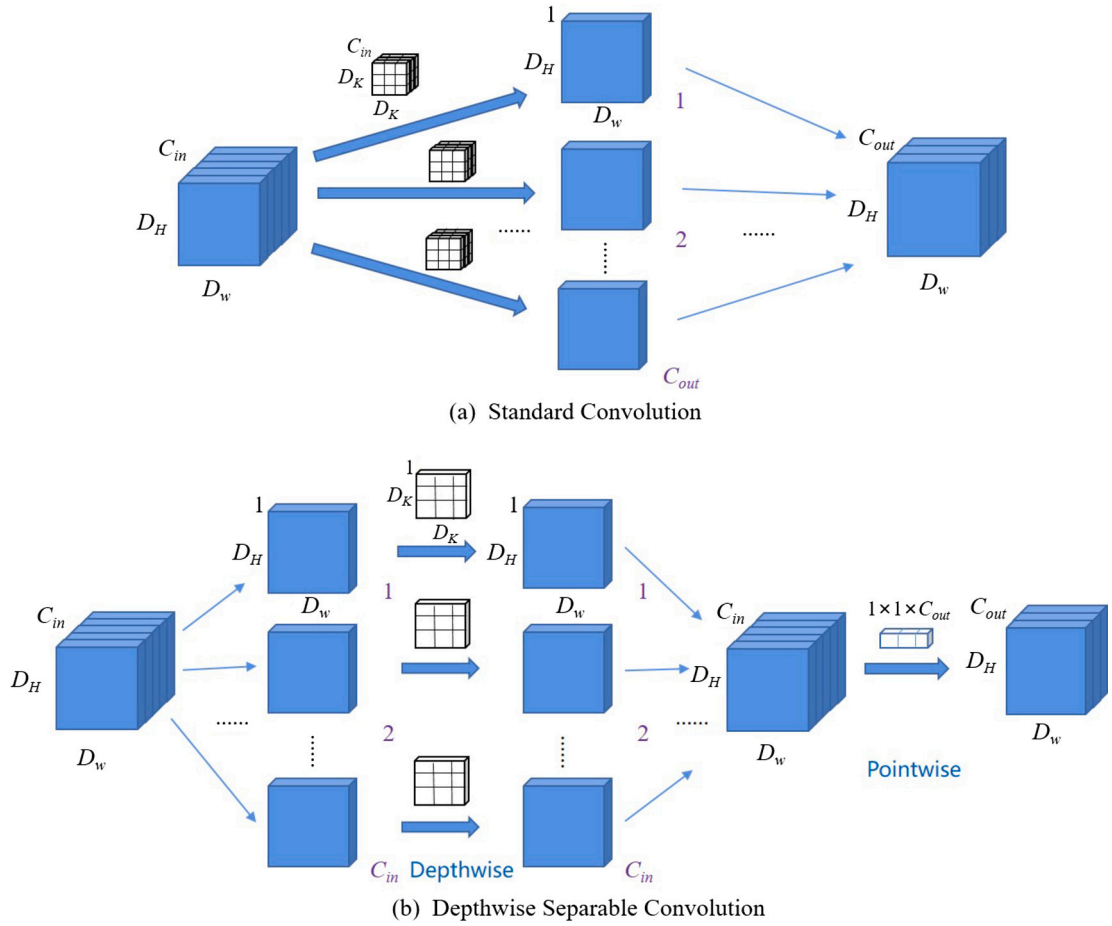


Fig. 1. The composition of standard convolution and depthwise separable convolution.

$$\frac{D_k \times D_k \times C_{in} + C_{in} \times C_{out}}{D_k \times D_k \times C_{in} \times C_{out}} = \frac{1}{C_{out}} + \frac{1}{D_k^2} \quad (1)$$

According to Exp. (1), we find that the calculation amount of the depthwise separable convolution is significantly reduced compared with that of the standard convolution.

3.4. Comparison between ELU and ReLU

The ELU function is defined as follows:

$$\begin{cases} x & \text{if } x > 0 \\ \alpha * (e^x - 1) & \text{if } x < 0 \end{cases} \quad (2)$$

ReLU Function: The ReLU function is defined as follows:

$$\begin{cases} \maxvalue & \text{if } x \geq \maxvalue \\ x & \text{if } x > 0 \\ \alpha * (x - \text{threshold}) & \text{otherwise} \end{cases} \quad (3)$$

Comparison and Analysis: The ELU function introduces an exponential term, allowing it to model more complex nonlinear relationships in the region where $x \leq 0$, which is not present in the simple linear behavior of the ReLU function. Moreover, the derivative of the ELU function for $x \leq 0$ is an exponential function, while the derivative of the ReLU function for $x \leq 0$ is 0. Consequently, the ELU function has a nonzero gradient in the negative region, which prevents the vanishing gradient problem. However, the ReLU function has a gradient of zero in the negative region, which may lead to the vanishing gradient.

In summary, the ELU function outperforms the ReLU function for modeling complex nonlinear data patterns. This is because the ELU function incorporates an exponential term for negative values, thereby enhancing its ability to capture complex nonlinear patterns. In addition, the ELU function has the advantage of a stable derivative that does not disappear in the negative region.

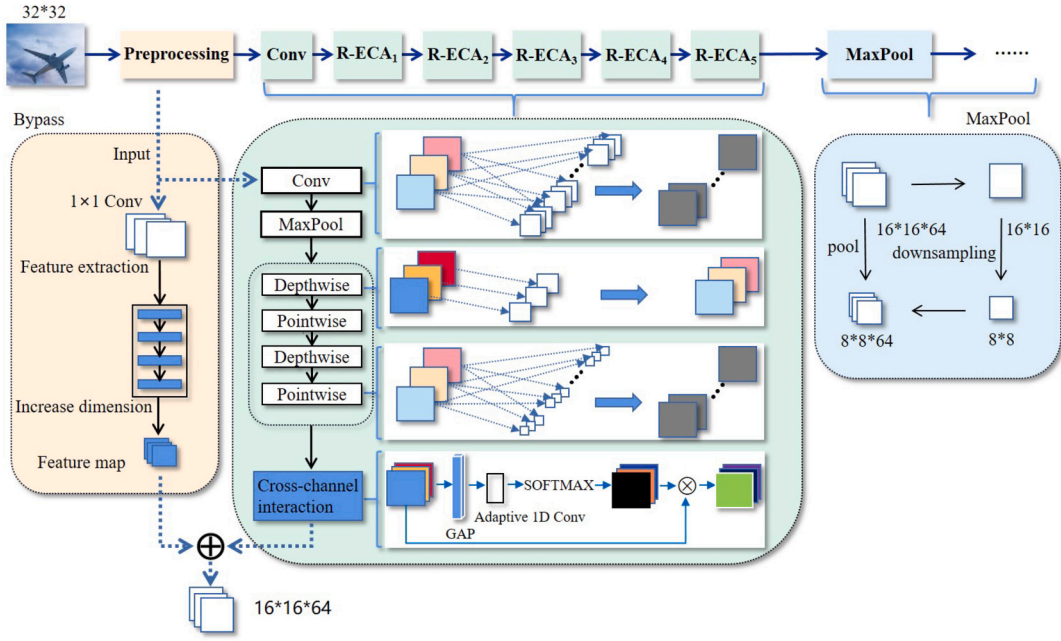


Fig. 2. L-Net network structure.

This property prevents the vanishing gradient problem, leading to improved training and optimization for deep neural networks. Depending on the circumstances, the ELU function may be more suitable than the ReLU function for DL tasks.

4. Construction of L-Net

In this section, we discuss the design concept of the R-ECA module and how it was used to construct our lightweight convolutional neural network, L-Net. L-Net is based on a depthwise separable convolution and comprises of five R-ECA modules. Each module includes a bypass mechanism, a cross-channel interaction, and multiple depthwise separable convolutions. The network structure of L-Net is presented in Fig. 2.

The bypass mechanism is a process that includes the expansion of the input dimensionality and the storage of the resulting expanded data. In this process, the non-expanded data undergoes multiple depthwise separable convolution operations, introducing a series of transformations. Following these convolutional operations, the data is subjected to processing to facilitate cross-channel interaction, enhancing the model's ability to capture complex patterns. Subsequently, the output of the cross-channel interaction, which now holds enriched features capturing relationships between different channels, is carefully combined with the originally expanded data obtained through the bypass mechanism. This combination ensures that the model benefits from both the high-level abstractions learned from the convolutional operations and the enriched contextual information derived from cross-channel interactions. The comprehensive integration of these components contributes to the model's overall effectiveness in handling complex data representations.

4.1. Bypass mechanism

When increasing the depth of a network, a phenomenon known as gradient vanishing can occur, leading to an unexpected increase in the training set loss. As a result, shallow networks might perform better than deep networks. To resolve this issue, it is crucial to transfer the features learned in the lower layers to the upper layers, ensuring that deep networks maintain performance levels at least on par with that of shallow ones. Additionally, traditional residual modules have fixed structures and cannot be added to the backbone network as required. To address this limitation, we propose a bypass mechanism inspired by the ResNet architecture. This mechanism involves increasing the input dimension using a dimensionality augmentation module, and then adding it to the result after convolution through n convolutional layers. This flexible approach allows us to add convolutional operations to each bypass module to achieve plug-and-play functionality. Moreover, our mechanism can partially mitigate issues such as gradient disappearance. Our bypass mechanism is based on the ResNet architecture, with depthwise separable convolution serving as the basic unit of the convolution layer (illustrated in Fig. 3). The input, denoted X_I , undergoes dimensionality augmentation and is subsequently combined with the output of n depthwise separable convolutions to obtain the final output. The bypass mechanism is represented as follows:

$$W_I X_I \oplus (W_{dp} (W_{dp} \dots (W_{dp} X_I) \dots)) \quad (4)$$

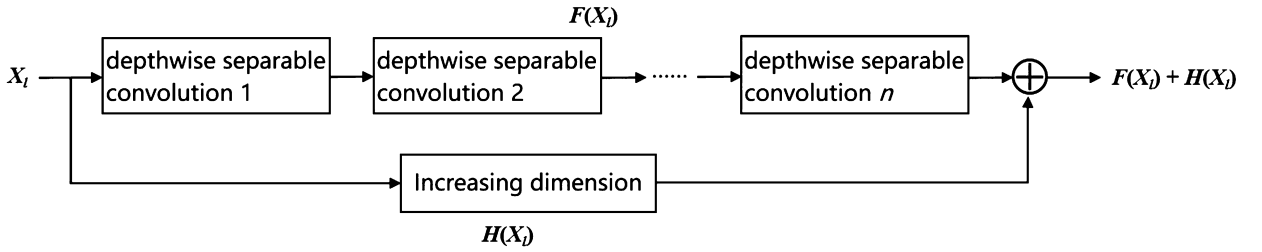


Fig. 3. The Bypass Mechanism.

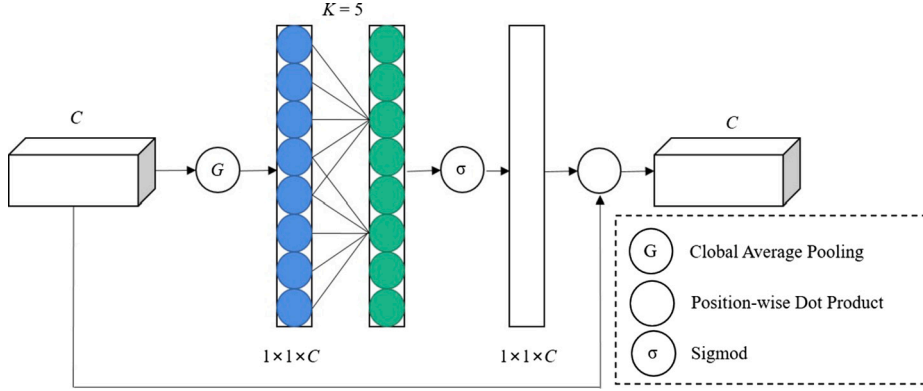


Fig. 4. Cross-channel Interaction of the Attention Mechanism.

The bypass mechanism in this process comprises n depthwise separable convolution modules and a dimension-raise module. W_l and W_{dp} refer to a 1×1 convolution operation and depthwise separable convolution operation, respectively. The depthwise convolution core size in the depthwise separable convolution module is 3×3 . The initial step size for all operations was set to 1. The dimension-raise module uses a convolution kernel size of 1×1 . Given an input X_l , the following operations are performed: First, the dimension of X_l is increased by evaluating $H(X_l)$, where H is a dimension-raising function of the branch pathway. Simultaneously, a series of depthwise separable convolution operations are applied to X_l , denoted as $F(X_l)$. Finally, the output of the bypass mechanism is obtained by adding together $H(X_l)$ and $F(X_l)$.

4.2. Cross-channel interaction

To address the issue of information isolation, blocking, and non-circulation among the depthwise separable convolution channels, we propose a cross-channel interaction mechanism. The working principle of this mechanism is illustrated in Fig. 4, and can be summarized as follows: First, the input feature map goes through a global average pooling operation to generate a feature map with $D_H = 1$ and $D_W = 1$, effectively reducing it to only the channel dimension. This feature map is then transformed into a long strip. After the convolution operation on a kernel of size one, each layer's channel interacts with the channel of the next layer, and shares its weight through a sigmoid function. Finally, the weight of the input feature map is multiplied by that of the processed feature map, which is added to the feature map.

4.3. R-ECA module

The R-ECA module enhances CNN performance by integrating bypass and cross-channel interaction mechanisms. The workflow of the module is as follows:

$$W_l X_l \oplus X_l \otimes \text{sig}((W_l X_l \oplus (W_{dp}(W_{dp} \dots (W_{dp} X_l) \dots))) \quad (5)$$

Here, X_l represents the input and W_l represents a 1×1 convolution operation. The model incorporates three operations: W_{1d} for one-dimensional convolution, W_{GAP} for global pooling, and W_{dp} for depthwise separable convolution. The process begins with a convolution operation applied to the input X_l , resulting in $F(X_l)$. Then, global average pooling is applied to $F(X_l)$, reducing the feature map to 1 for D_H and D_W dimensions, while preserving only the channel dimension. Subsequently, a cross-channel interaction is achieved between adjacent layers by performing a 1×1 convolution operation with a shared weight. The input X_l is multiplied with the processed feature map weight using sigmoid, resulting in $F'(X_l)$. Simultaneously, the branch pathway utilizes a dimension upgrading module to upgrade the input X_l and produce $H(X_l)$ with the same dimensions as $F'(X_l)$. Combining $H(X_l)$ and $F'(X_l)$ generates X_{l+1} . Fig. 5 illustrates the workflow of the R-ECA module.

Regarding the workflow of R-ECA, we express it in the form of pseudocode, as shown in Algorithm 1.

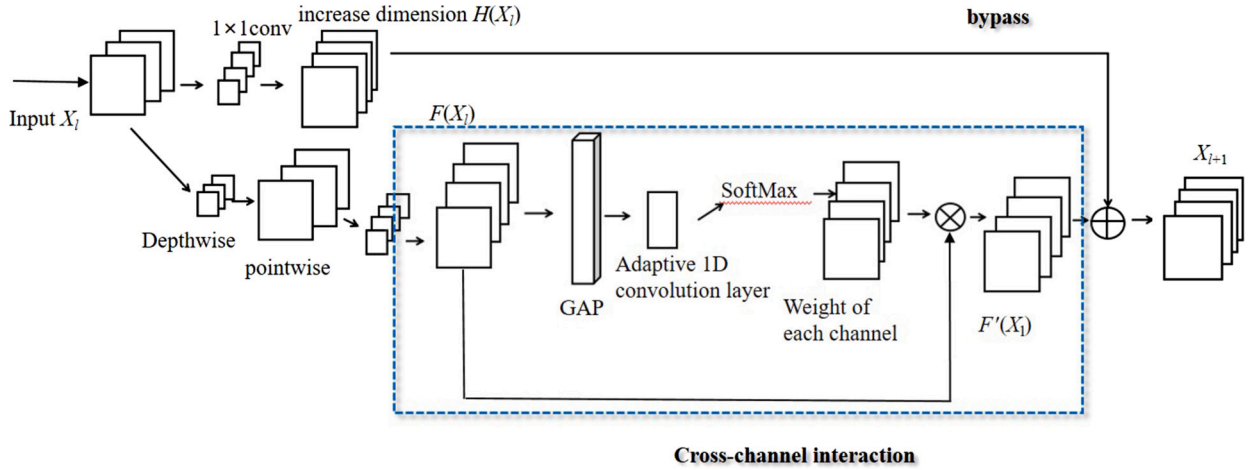


Fig. 5. The R-ECA Module.

Algorithm 1 REAC.

Input: $D = \{(x^{(n)}, y^{(n)})\}_{n=1}^N, L$
 Reorder the samples in the training set.
for $n = 1$ to L **do**
 Selecting $(x^{(n)}, y^{(n)})$ from the training set D .
 $H(X_i)$
 Forward Compute the input X_i^p for each layer ($1 < p < L$)
 if filters of $p + 1 >$ filters of p **then**
 $X_{i+1} \leftarrow X_i \oplus X_i^p$
 else
 $X_i \leftarrow X_i \otimes \text{sig}((W_i X_i \oplus (W_{dp}(W_{dp} \dots (W_{dp} X_i) \dots)))$
 end if
end for
Output: Output result X_{i+1}

4.4. L-Net

The creation of L-Net involves incorporating the R-ECA module mentioned in Subsection 4.3 into the backbone network formed by depthwise separable convolution. The detailed model structure of the L-Net is presented in Table 2. First, the input feature map is passed through a 3×3 convolution layer. As it passes through a sequence of depthwise separable convolution basic units, R-ECA blocks, and additional depthwise separable convolution basic units, the feature map size and number of output channels change in the following order: one, one, two, one, two, another one, five, one (final), and two. To enhance feature extraction and model convergence, the output obtained after convolution processing outside the L-Net should undergo batch-normalization (or BN) layer normalization and employ the ELU nonlinear activation function to improve its nonlinear expression capability. Notably, the use of the ELU activation function is recommended over the ReLU function.

The following Algorithm 2 is the basic process of L-Net.

Algorithm 2 L-Net.

Input: repeat
Input: $D = \{(x^{(n)}, y^{(n)})\}_{n=1}^N, L$, learn rate α
 Reorder the samples in the training set.
for $n = 1$ to L **do**
 Selecting $(x^{(n)}, y^{(n)})$ from the training set D .
 $H(X_i)$
 Forward Compute the input X_i^p for each layer ($1 < p < L$)
 if filters of $p + 1 >$ filters of p **then**
 $X_{i+1} \leftarrow X_i \oplus X_i^p$
 else
 $X_i \leftarrow X_i \otimes \text{sig}((W_i X_i \oplus (W_{dp}(W_{dp} \dots (W_{dp} X_i) \dots)))$
 end if
 Backpropagation computes the error σ^l for each layer's parameters.
 $W = W - \alpha \frac{\partial J}{\partial W}$
end for
Input: until J no longer decreases
Output: W

Table 2
The Model Structure of L-Net.

Input size	Convolution operation	Number of output channels	Strides
$32 \times 32 \times 3$	conv	32	2
$16 \times 16 \times 32$	dp1	32	1
$16 \times 16 \times 32$	R-ECA1	64	2
$16 \times 16 \times 64$	dp2	128	2
$8 \times 8 \times 128$	dp3	128	1
$8 \times 8 \times 128$	R-ECA2	128	2
$8 \times 8 \times 128$	dp4	256	1
$4 \times 4 \times 256$	dp5	256	2
$4 \times 4 \times 256$	R-ECA3	256	2
$4 \times 4 \times 256$	dp6	512	1
$2 \times 2 \times 512$	dp7	512	1
$2 \times 2 \times 512$	dp8	512	1
$2 \times 2 \times 512$	dp9	512	1
$2 \times 2 \times 512$	dp10	512	1
$2 \times 2 \times 512$	dp11	512	1
$2 \times 2 \times 512$	R-ECA4	512	2
$2 \times 2 \times 512$	dp12	1024	1
$1 \times 1 \times 1024$	dp13	1024	2
$1 \times 1 \times 1024$	R-ECA5	1024	2

5. Experiment and result analysis

5.1. Dataset preparation

The CIFAR-10 dataset [1,17] was employed for training and testing the model in this experiment. This dataset comprises red-green blue (or RGB) color pictures of common objects and was compiled by Krizhevsky and Sutskever. The dataset comprised 10 categories: aircraft, automobiles, birds, cats, deer, dogs, frogs, horses, ships, and trucks. Each image measured 32×32 pixels in size. The dataset includes 50000 training samples and 10000 test pictures. Moreover, data augmentation techniques were used to modify the original dataset by applying geometric and pixel-level transformations.

5.2. Experimental environment

We conducted our experiments using the Python programming language and TensorFlow DL framework. The computations were performed on a powerful NVIDIA RTX 3080 GPU (with 24 GB of video memory and 32 GB of RAM available) for training and testing. To monitor the performance of the model and track different indicators during training, we employed the visualization tool Matplotlib. During each training iteration, the model exhibited a remarkable performance improvement, as reflected by the decrease in training loss and continuous enhancement in the image distortion rate. Furthermore, the accuracy of the image recognition gradually improved over time.

5.3. Performance comparison

5.3.1. Accuracy, loss, parameter, and weight size comparison

We compared several models, including L-Net, VGG11 [30], AlexNet [18], SqueezeNet [16], and MobileNetV1 [12], etc. Initially, we configured the number of epochs to 1000 and followed the training process for each model. We began by setting the initial learning rate to 0.03 and the batch size to 64. This allowed us to reduce the loss value during the first 400 epochs and obtain the initial loss value. Notably, the batch size determines the number of samples selected for one training, thereby influencing the model optimization and speed. Therefore, it was adjusted based on the model parameters and GPU memory capacity. The learning rate was then decreased to 0.003 for the next 400 epochs, and further reduced to 0.0003 for the subsequent 200 epochs to obtain the final loss value for the training set. We utilized this process to train both models and present their parameter file sizes, model precision, model file size, and loss functions in Table 3. Although the L-Net model is smaller than some models, it outperforms the representative MobileNetV1 with a higher accuracy of 9.14% and a lower loss function of 7.10%. However, its overall performance still falls behind models such as MobileNetV2, the Shufflenet series, and ResNet20. These results indicate that while L-Net excels in reducing the loss function and improving accuracy, there is still room for improvement, particularly when compared to the mentioned advanced models.

5.3.2. mAP comparison

While evaluating the object recognition models, the average precision (AP) was determined by considering both precision (P) and recall (R). To assess the overall performance of the object recognition model, the mean average precision (mAP) is computed using a specific formula.

$$P = TP / (TP + FP) \quad (6)$$

Table 3
Comparison of the models.

Model	Accuracy	Loss	Param(million)	Model File Size(MB)
VGG11 [30]	0.7853	1.005	33.64	128.38
AlexNet [18]	0.7199	1.692	60	67.53
SqueezeNet [16]	0.7204	1.5742	1.2	8.57
ResNet20 [11]	0.8639	0.4678	0.2	2.45
MobileNetV1 [12]	0.7264	1.4731	3.2	12.56
MobileNetV2 [29]	0.8776	0.4361	0.420	1.58
ShuffleNetV1 [42]	0.8460	0.5917	6.04	26.08
ShuffleNetV2 [26]	0.8514	0.5706	0.36	1.42
GoogLeNet [32]	0.8048	0.9531	6.013	23.40
PeeleNet [35]	0.8470	0.5335	6.751	25.89
EfficientNet [34]	0.5200	1.5467	4.062	15.97
L-Net	0.8299	0.5572	3.9	15.32

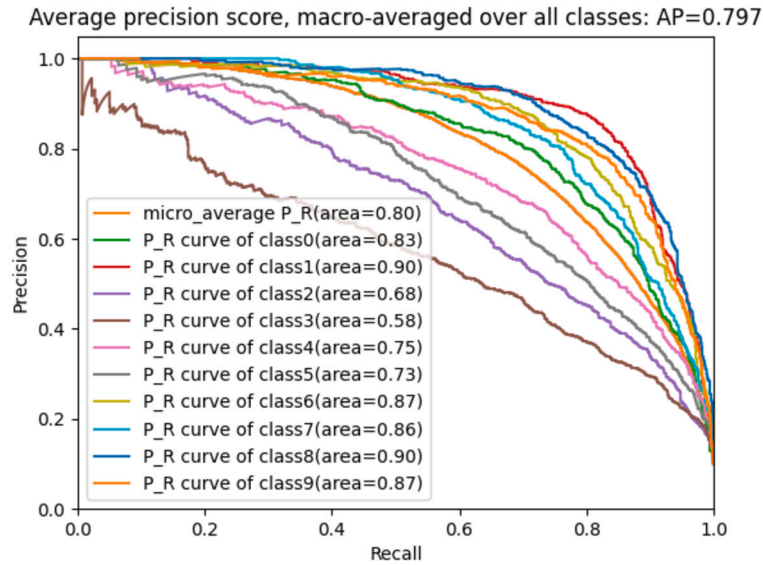


Fig. 6. The AP of AlexNet.

$$R = TP / (TP + FN) \quad (7)$$

This study introduces the concept of Intersection over Union (IOU), which calculates the ratio between the intersection and union of the detection and the ground-truth frames. The model utilizes a predefined IOU threshold of 0.5. If the IOU exceeds the threshold and the detection frame is correct, it is referred to as a true positive (TP). If the IOU is below the threshold and the detection frame is incorrect, it is denoted as a false positive (FP). Finally, If the ground truth frame is true and correct but the model fails to detect it, it is classified as a false negative (FN).

We followed the method described previously to obtain an AP map for detecting defects in 12 different models. The shaded region on the map indicates the calculated AP with the recall rate on the x -axis, and the precision rate of a specific model type on the y -axis. Figs. 6–17 depict the AP diagrams of the 12 DL models.

Table 4 presents the AP values for each model type. Compared to other models, L-Net achieves relatively higher AP values in target recognition. These results indicate that within the CIFAR-10 dataset, L-Net exhibits relatively higher precision in predicting each category.

The mean average precision (mAP) values for the models can be obtained as follows:

$$mAP = \frac{\sum_{i=1}^k AP_i}{k} \quad (8)$$

Table 5 lists the mAP values for the 12 models. L-Net exhibited higher mAP values than AlexNet, VGG11, SqueezeNet, EfficientNet, GoogLeNet, and MobileNetV1, with These results demonstrate the effectiveness of the proposed method.

Therefore, the above experimental results support the feasibility and effectiveness of the proposed L-Net.

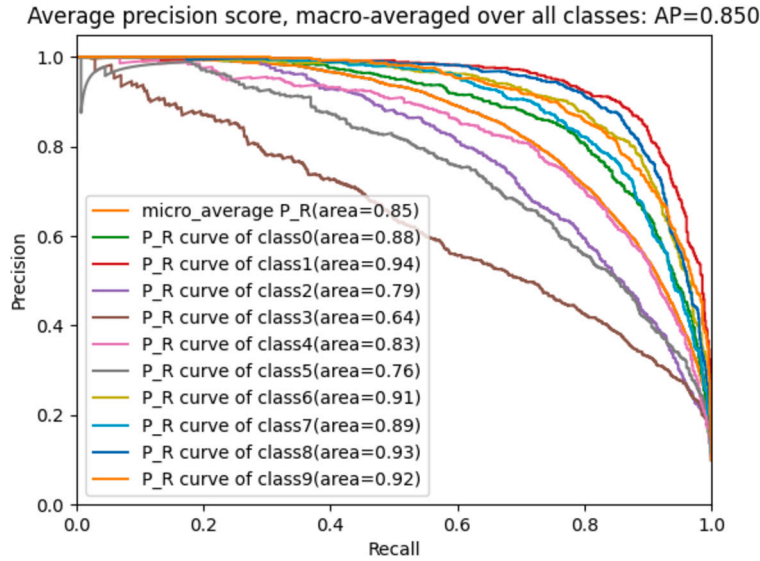


Fig. 7. The AP of VGG11.

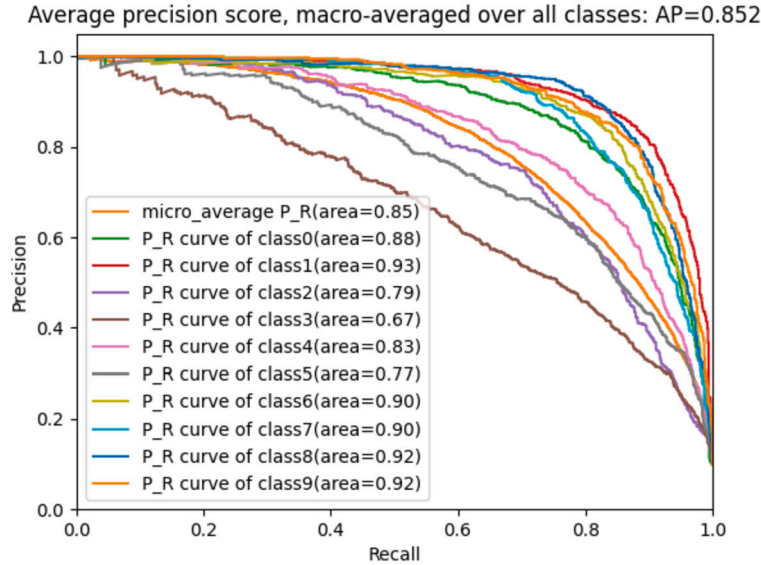


Fig. 8. The AP of MobileNetV1.

5.3.3. Comparison of ELU and ReLU

In this study, the rectified linear unit function (ReLU) was replaced with the exponential linear unit function (ELU) to improve the network's non-linear expression capability and training speed. For comparative purposes, the epoch was set to 300. Fig. 18 illustrates the distinct performances of L-Net when using the ELU and ReLU functions. It is evident that the use of the ELU function significantly accelerates the convergence speed. Additionally, the time required to reach convergence was recorded, with L-Net using the ELU function achieving convergence in 2499.033 seconds, compared to 2603.322 seconds for the ReLU function. The utilization of the ELU function resulted in a noteworthy reduction in convergence time.

6. Application of this scheme in agriculture

In this section, we describe the use of L-Net to detect apple leaf diseases. Fig. 19 presents examples of common apple leaf diseases, including apple Scab, black Rot, cedar apple rust, mosaic, alternaria blotch, brown spot, grey spot, rust. To ensure the completeness of the data, we also added a dataset of healthy apple leaves. The entire process encompasses selecting and preprocessing apple leaf diseases, extracting features using DL models, analyzing the features, and generating prediction suggestions.

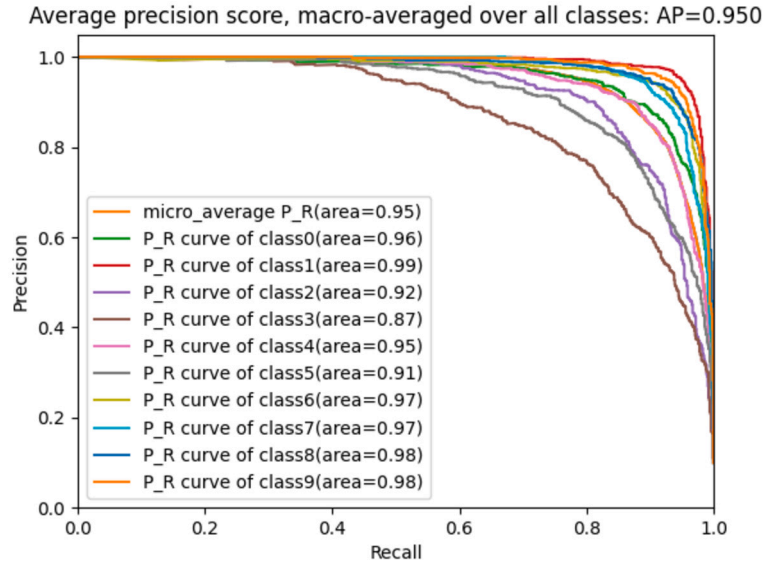


Fig. 9. The AP of MobileNetV2.

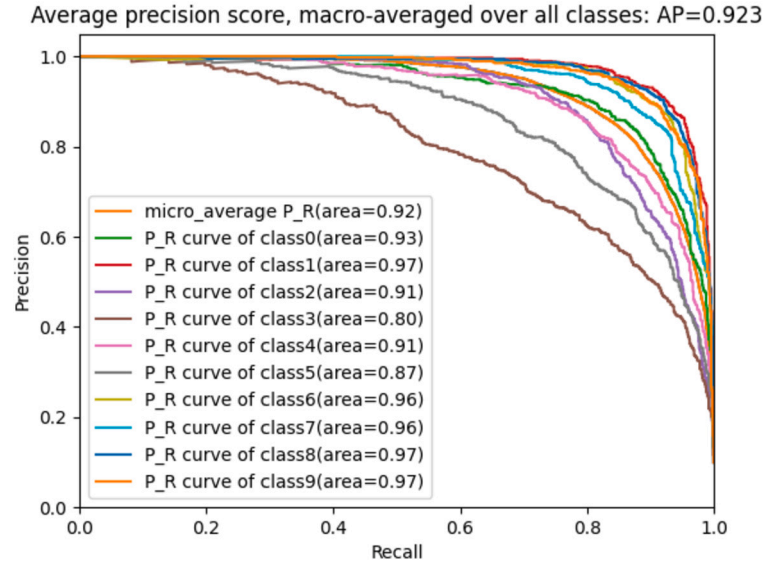


Fig. 10. The AP of ShuffleNetV1.

6.1. Dataset preparation

We considered eight common apple leaf diseases dataset and a healthy apple leaf dataset, totaling approximately 27,000 images which were adjusted uniformly to 227×227 pixels to make them suitable for model training. Partial data was selected from the PlantVillage dataset, which is publicly available [31]. LabelImg, an image annotation tool, was used to label lesions in the images. We randomly divided the data into training and validation sets in an 9 : 1,8 : 2 and 7 : 3 ratio.

6.2. Performance comparison

During the training process, we carefully select multiple essential hyperparameters to optimize the performance of the DL models. These choices were made to strike a balance between computational efficiency and ensuring the convergence of the model to an optimal state. In terms of performance metrics, we chose macro recall, macro precision, and macro F1 score. Macro recall is a performance metric used to evaluate the performance of a classification model. It measures the average recall across all classes. To calculate macro recall, we first calculate the recall for each class, which is the number of true positive predictions divided by the total number of actual positive samples for that class. Then, we take the average of these recall values. This approach gives equal weight

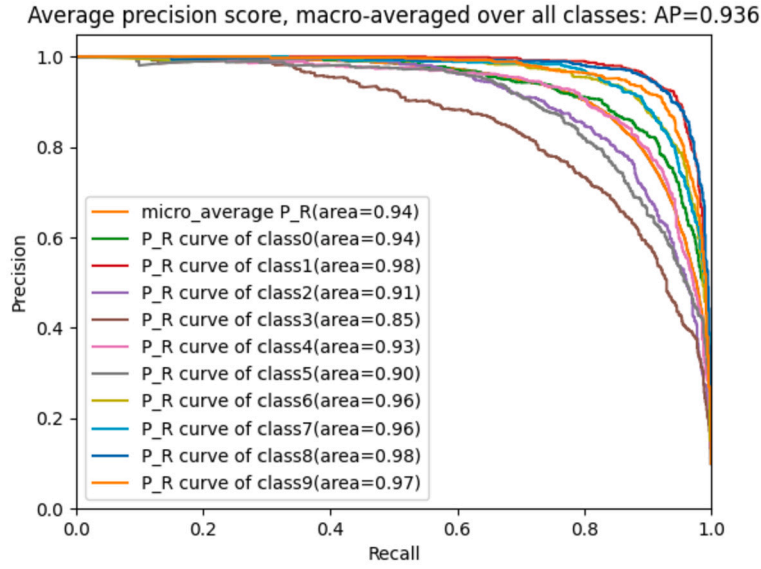


Fig. 11. The AP of ShuffleNetV2.

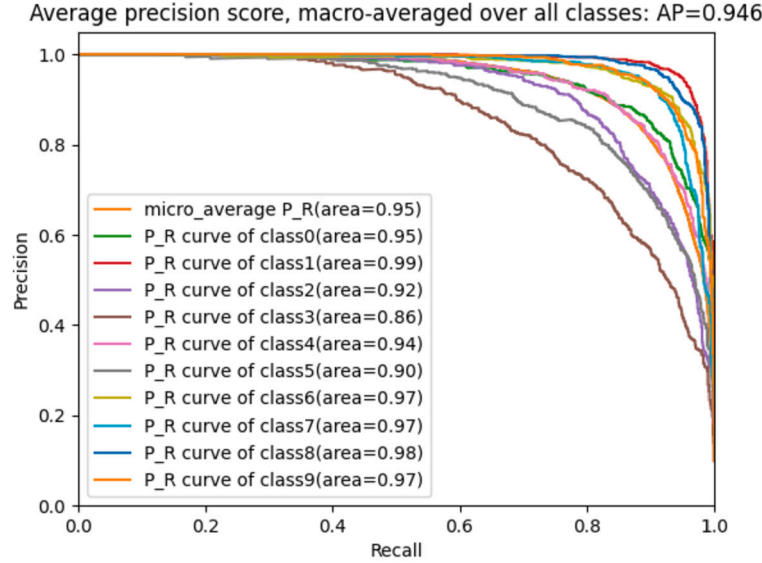


Fig. 12. The AP of ResNet20.

to each class, regardless of the number of samples in each class. Macro recall is suitable for multi-class scenarios as it provides a fair evaluation of performance for each class. Macro precision is a performance metric used to evaluate the performance of a classification model. It measures the average precision across all classes. To calculate macro precision, we first calculate the precision for each class, which is the number of true positive predictions divided by the total number of predicted positive samples for that class. Then, we take the average of these precision values. This approach gives equal weight to each class, regardless of the number of samples in each class. Macro precision is suitable for multi-class scenarios as it provides a fair evaluation of performance for each class.

6.3. Performance comparison

In this section, we partition the data into training and testing sets using the ratios of 9:1, 8:2, and 7:3, respectively. We then record and compare the results of different models on these datasets. We compared all the models that were recorded in Section 5. Tables 6, 7, and 8 display the performance of different models under the training and validation set ratios of 9:1, 8:2, and 7:3 respectively.

To gauge the disparities between different models, we computed the standard deviation of performance metrics across various splitting ratios. This involved selecting different splitting ratios (9:1, 8:2, and 7:3) to partition the data into training and test sets.

Table 4
The *AP* Values of each model type.

Model	Class1APvalue	Class2APvalue	Class3APvalue	Class4APvalue	Class5APvalue
AlexNet [18]	0.83	0.90	0.68	0.58	0.75
VGG11 [30]	0.88	0.94	0.79	0.64	0.83
MobileNetV1 [12]	0.88	0.93	0.79	0.67	0.83
MobileNetV2 [29]	0.96	0.99	0.92	0.87	0.95
ShuffleNetV1 [42]	0.93	0.97	0.91	0.80	0.91
ShuffleNetV2 [26]	0.94	0.98	0.91	0.85	0.93
ResNet20 [11]	0.95	0.99	0.92	0.86	0.94
GoogLeNet [32]	0.92	0.86	6.87	0.74	0.89
PeeleeNet [35]	0.94	0.98	0.91	0.86	0.93
EfficientNet [34]	0.54	0.77	0.26	0.34	0.50
SqueezeNet [16]	0.82	0.90	0.72	0.61	0.80
L-Net	0.93	0.97	0.87	0.75	0.88

Model	Class6APvalue	Class7APvalue	Class8APvalue	Class9APvalue	Class10APvalue
AlexNet [18]	0.73	0.87	0.86	0.90	0.87
VGG11 [30]	0.76	0.91	0.89	0.93	0.92
ResNet20 [11]	0.90	0.97	0.97	0.98	0.97
MobileNetV1 [12]	0.77	0.90	0.90	0.92	0.92
MobileNetV2 [29]	0.91	0.97	0.97	0.98	0.98
ShuffleNetV1 [42]	0.87	0.96	0.96	0.97	0.97
ShuffleNetV2 [26]	0.90	0.96	0.96	0.98	0.97
ResNet20 [11]	0.95	0.99	0.92	0.86	0.94
GoogLeNet [32]	0.83	0.94	0.81	0.93	0.70
PeeleeNet [35]	0.90	0.96	0.96	0.98	0.98
EfficientNet [34]	0.44	0.72	0.66	0.72	0.70
SqueezeNet [16]	0.82	0.90	0.72	0.61	0.80
L-Net	0.84	0.95	0.95	0.97	0.96

Table 5
The *mAP* Values.

Model	mAP value
AlexNet [18]	0.797
VGG11 [30]	0.850
MobileNetV1 [12]	0.852
MobileNetV2 [29]	0.950
ShuffleNetV1 [42]	0.923
ShuffleNetV2 [26]	0.936
ResNet20 [11]	0.946
GoogLeNet [12]	0.849
PeeleeNet [35]	0.939
EfficientNet [34]	0.563
SqueezeNet [16]	0.810
L-Net	0.906

Table 6
Comparison in training and validation set ratios of 9:1.

Model	Accuracy	Marco Precision	Marco Recall	Marco F1 score
AlexNet [18]	0.893	0.909	0.894	0.892
VGG11 [30]	0.802	0.827	0.803	0.790
MobileNetV1 [12]	0.919	0.932	0.919	0.915
MobileNetV2 [29]	0.935	0.947	0.936	0.933
ShuffleNetV1 [42]	0.872	0.897	0.872	0.860
ShuffleNetV2 [26]	0.934	0.947	0.935	0.932
ResNet20 [18]	0.814	0.827	0.815	0.809
GoogLeNet [12]	0.931	0.941	0.931	0.929
PeeleeNet [35]	0.880	0.902	0.880	0.873
EfficientNet [34]	0.864	0.890	0.865	0.854
SqueezeNet [11]	0.928	0.939	0.929	0.926
L-Net	0.923	0.937	0.924	0.920

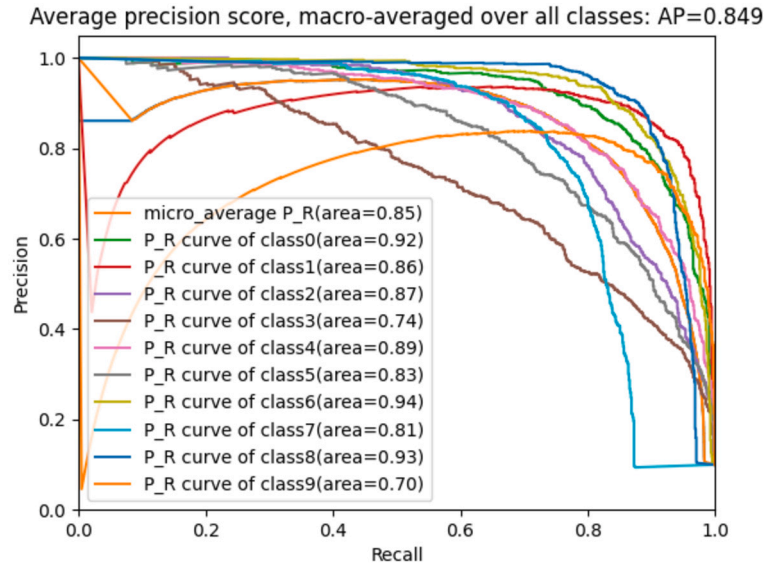


Fig. 13. The AP of GoogLeNet.

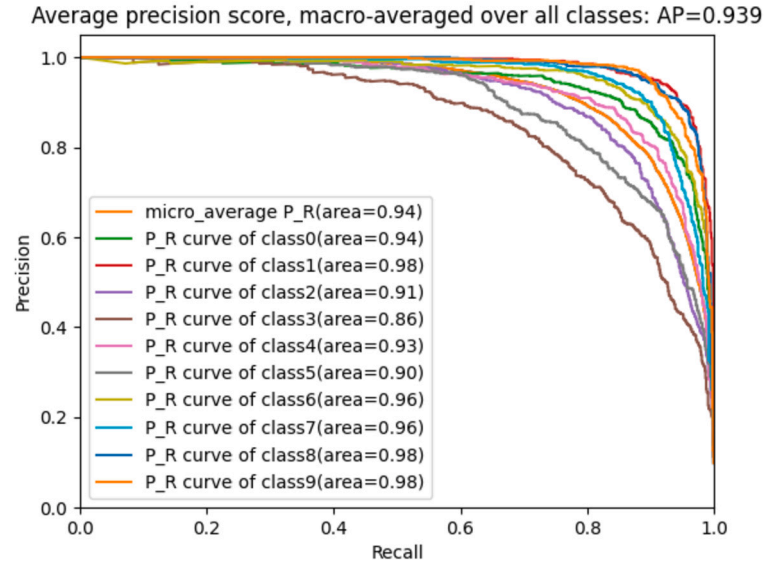


Fig. 14. The AP of PeleeNet.

Table 7

Comparison in training and validation set ratios of 8:2.

Model	Accuracy	Marco Precision	Marco Recall	Marco F1 score
AlexNet [18]	0.869	0.889	0.872	0.870
VGG11 [30]	0.793	0.809	0.805	0.785
MobileNetV1 [12]	0.909	0.936	0.912	0.911
MobileNetV2 [29]	0.921	0.941	0.923	0.924
ShuffleNetV1 [42]	0.848	0.885	0.852	0.848
ShuffleNetV2 [26]	0.917	0.940	0.920	0.920
ResNet20 [18]	0.760	0.783	0.763	0.759
GoogLeNet [32]	0.907	0.929	0.910	0.910
PeleeNet [35]	0.875	0.900	0.875	0.871
EfficientNet [34]	0.860	0.885	0.862	0.860
SqueezeNet [11]	0.904	0.925	0.907	0.907
L-Net	0.910	0.928	0.912	0.912

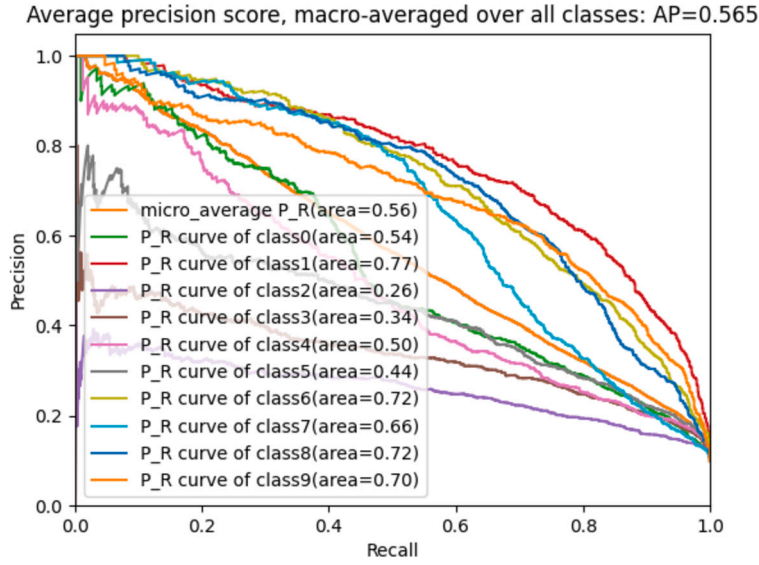


Fig. 15. The AP of EfficientNet.

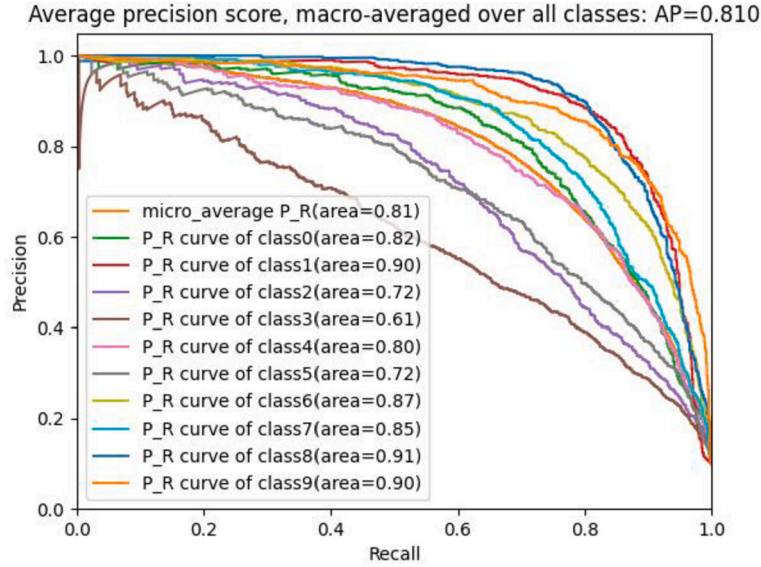


Fig. 16. The AP of SqueezeNet.

Table 8

Comparison in training and validation set ratios of 7:3.

Model	Accuracy	Marco Precision	Marco Recall	Marco F1 score
AlexNet [18]	0.855	0.871	0.856	0.853
VGG11 [30]	0.793	0.806	0.793	0.782
MobileNetV1 [12]	0.910	0.931	0.912	0.909
MobileNetV2 [29]	0.908	0.931	0.910	0.908
ShuffleNetV1 [42]	0.842	0.886	0.846	0.839
ShuffleNetV2 [26]	0.908	0.925	0.910	0.909
ResNet20 [18]	0.722	0.760	0.724	0.722
GoogLeNet [32]	0.909	0.924	0.910	0.909
PeleeNet [35]	0.834	0.864	0.837	0.833
EfficientNet [34]	0.841	0.872	0.844	0.839
SqueezeNet [11]	0.892	0.907	0.893	0.891
L-Net	0.910	0.928	0.910	0.908

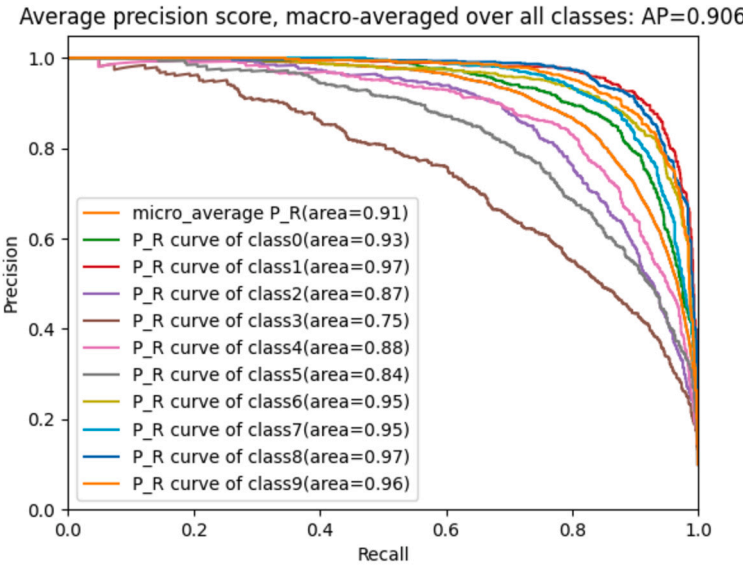


Fig. 17. The AP of L- Net.

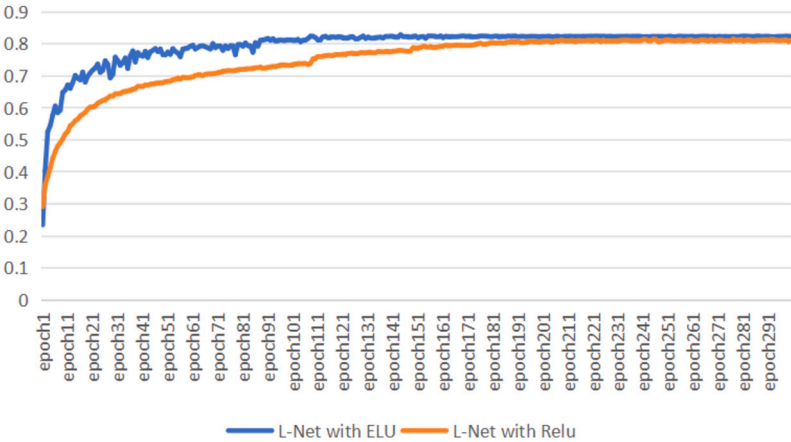


Fig. 18. Comparison of ELU and ReLU.

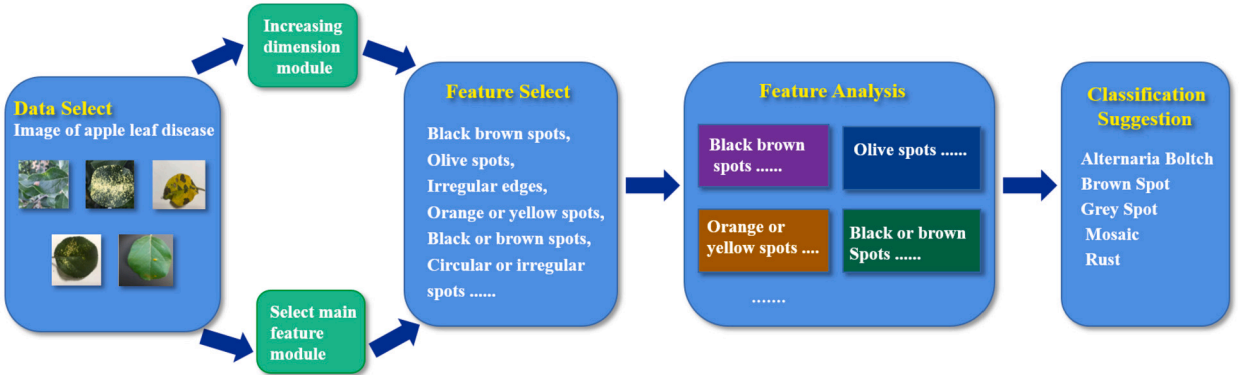


Fig. 19. Prediction of apple leaf diseases.

Table 9
Comparison of standard deviation.

Model	SD of Marco Precision	SD of Marco Recall	SD of Marco F1 score
AlexNet [18]	0.0190	0.0191	0.0196
VGG11 [30]	0.0114	0.0191	0.004
MobileNetV1 [12]	0.0026	0.004	0.0031
MobileNetV2 [29]	0.0081	0.013	0.0125
ShuffleNetV1 [42]	0.0067	0.0136	0.0115
ShuffleNetV2 [26]	0.0112	0.0126	0.0115
ResNet20 [18]	0.034	0.0457	0.0437
GoogLeNet [32]	0.0087	0.0121	0.0095
PeeleNet [35]	0.0214	0.0235	0.0225
EfficientNet [34]	0.0052	0.0114	0.0108
SqueezeNet [11]	0.016	0.0181	0.0175
L-Net	0.0052	0.0076	0.0061

Subsequently, each model was trained using the training set, and its performance was evaluated on the corresponding test set. Performance metrics such as accuracy, precision, recall, and F1 score were then calculated for each model on different test sets. The standard deviation of these performance metrics was computed for each model across different test sets. This approach allows us to assess the actual differences between models by quantifying the standard deviation of their performance metrics across diverse splitting ratios. The Table 9 details the standard deviations of macro precision, macro recall, and macro F1 score for each method.

According to Table 9, it can be observed that the proposed L-Net exhibits smaller standard deviations in macro recall, macro precision, and macro F1 score compared to other models, with only a slightly larger value than MobileNetV1. This suggests that L-Net demonstrates relatively consistent performance across different data set splits, indicating minimal variation in model performance under various training and testing set ratios. Moreover, the model shows stability in the face of fluctuations or changes in the data, demonstrating reliable performance even when faced with varying data scales. The ability of the model to maintain consistent performance across datasets of different sizes implies robust generalization to a broader range of data distributions.

7. Conclusion

This study proposes L-Net, a lightweight CNN designed for devices with low computing power. L-Net features an optimized network structure and fewer parameters, making it computationally efficient and well suited for resource-constrained environments. It introduces the R-ECA module, which combines simplified residual learning and an attention mechanism to address challenges related to channel interaction disparity and vanishing gradient in lightweight DL. Moreover, L-Net enhances its nonlinear expressive power by replacing the ReLU function with the ELU function. The nonzero gradient of the ELU function in the negative region alleviates the vanishing gradient problem, thereby improving the network's ability to fit nonlinear patterns, model accuracy, and stability. Numerous experimental results have demonstrated high accuracy, rapid model training convergence, and stability of L-Net.

The performance of L-Net was evaluated on diverse datasets, consisting of both public datasets and custom datasets, and compared against well-known neural networks. Although our model outperformed certain models on public datasets, there still exists a performance gap when compared to models such as MobileNetV2, ResNet20, and ShuffleNet. Nevertheless, given the model's parameter count of 3.9 million and achieving an accuracy of 83%, the overall performance can still be considered satisfactory. For our custom datasets, we partitioned them into varying ratios and determined the actual differences for each model. L-Net exhibited relatively consistent performance across different dataset splits, outperforming the majority of the compared models. This suggests that the model's performance exhibits minimal variation when subjected to different training and testing set ratios. Furthermore, the model displayed stability when confronted with data fluctuations or changes, showcasing reliable performance amidst diverse data scales. The model's capability to sustain consistent performance across datasets of varying sizes indicates its robust ability to generalize to diverse data distributions.

In the future, we plan to focus on enhancing the accuracy and robustness of L-Net by reducing the number of parameters. Additionally, we aim to incorporate additional performance indicators and optimize the model for deployment in mobile devices, thereby increasing its practicality and versatility.

CRediT authorship contribution statement

Hua Shen: Formal analysis, Methodology, Validation, Writing – original draft. **Zhiwei Wang:** Data curation, Methodology, Software, Writing – original draft. **Jixin Zhang:** Funding acquisition, Supervision, Validation, Writing – review & editing. **Mingwu Zhang:** Investigation, Supervision, Writing – review & editing, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgement

This work is supported in part by the National Natural Science Foundation of China (61702168, 62072134, 62002106, U2001205), and in part by the Green Industry Technology Leading Program of Hubei University of Technology (XJ2021000901).

References

- [1] Canadian Institute for Advanced Research (CIFAR). Cifar-10 dataset, <https://www.cs.toronto.edu/~kriz/cifar.html>, 2009.
- [2] Sheng Chen, Yang Liu, Xiang Gao, Zhen Han, Mobilefacenet: efficient cnns for accurate real-time face verification on mobile devices, in: Biometric Recognition: 13th Chinese Conference, CCBR 2018, Urumqi, China, August 11–12, 2018, Proceedings 13, Springer, 2018, pp. 428–438.
- [3] X. Chen, M.J. Wan, C. Ma, Q. Chen, G.H. Gu, Recognition of small targets in remote sensing image using multi-scale feature fusion-based shot multi-box detector, *Opt. Precis. Eng.* 011 (2021) 029.
- [4] X. Dai, X. Yuan, X. Wei, Data augmentation for thermal infrared object detection with cascade pyramid generative adversarial network, *Appl. Intell.* 52 (1) (2022) 967–981.
- [5] J. Deng, W. Dong, R. Socher, L.J. Li, K. Li, F.F. Li, Imagenet: a large-scale hierarchical image database, in: 2009 IEEE Conference on Computer Vision and Pattern Recognition, IEEE, Miami, FL, USA, 2009, pp. 248–255.
- [6] M. Everingham, L. Van Gool, C.K. Williams, J. Winn, A. Zisserman, The pascal visual object classes (voc) challenge, *Int. J. Comput. Vis.* 88 (2010) 303–338.
- [7] G. Ghiasi, T.Y. Lin, Q.V. Le, Nas-fpn, Learning scalable feature pyramid architecture for object detection, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2019, pp. 7036–7045.
- [8] R. Girshick, Fast r-cnn, in: Proceedings of the IEEE International Conference on Computer Vision, 2015, pp. 1440–1448.
- [9] R. Girshick, J. Donahue, T. Darrell, J. Malik, Rich feature hierarchies for accurate object detection and semantic segmentation, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2014, pp. 580–587.
- [10] K. He, X. Zhang, S. Ren, J. Sun, Spatial pyramid pooling in deep convolutional networks for visual recognition, *IEEE Trans. Pattern Anal. Mach. Intell.* 37 (9) (2015) 1904–1916.
- [11] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 770–778.
- [12] A.G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, H. Adam Mobilenets, Efficient convolutional neural networks for mobile vision applications, preprint, arXiv:1704.04861, 2017.
- [13] G. Huang, Z. Liu, L. Van Der Maaten, K.Q. Weinberger, Densely connected convolutional networks, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2017, pp. 4700–4708.
- [14] H. Huang, L. Lin, R. Tong, H. Hu, J. Wu, Unet 3+: A full-scale connected unet for medical image segmentation, arXiv, 2020.
- [15] Y.L. Huang, N. Li, Z.B. Liu, Y.H. Zhang, Improved lightweight convolutional networks for classification of grape diseased leaves, *J. Harbin Univ. Sci. Technol.* 28 (3) (2023).
- [16] F.N. Iandola, S. Han, M.W. Moskewicz, K. Ashraf, W.J. Dally, K. Keutzer Squeezenet, Alexnet-level accuracy with 50x fewer parameters and <0.5 mb model size, preprint, arXiv:1602.07360, 2016.
- [17] A. Krizhevsky, G. Hinton, Learning multiple layers of features from tiny images, in: Handbook of Systemic Autoimmune Diseases, vol. 1(4), 2009, pp. 32–36.
- [18] A. Krizhevsky, I. Sutskever, G.E. Hinton, Imagenet classification with deep convolutional neural networks, *Commun. ACM* 60 (6) (2017) 84–90.
- [19] Y. LeCun, L. Bottou, Y. Bengio, P. Haffner, Gradient-based learning applied to document recognition, *Proc. IEEE* 86 (11) (1998) 2278–2324.
- [20] K.S. Li, Z.S. Yang, Z.F. Jiang, J.C. Wang, H. Wang, Detection and identification of crop leaf diseases based on improved yolox-nano, *J. South China Agricult. Univ.* 44 (4) (2023) 593–603.
- [21] K.Y. Li, O. Ou, G.B. Liu, Z.F. Yu, L. Li, Target detection algorithm of remote sensing image based on improved yolov5, *J. Comput. Eng. Appl.* 59 (9) (2023).
- [22] N. Lin, T. Huang, P.L. Sun, Building extraction of high-resolution remote sensing imagery on optimized mask rcnn, *Remote Sens. Inf.* 003 (2022) 037.
- [23] M. Liu, J. Liu, L.J. Yin, Z.W. Kang, X. Ma, Martian image classification based on iterative pruning vggne, *Chin. J. Liquid Crystal Displays* 38 (4) (2023).
- [24] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.Y. Fu, A.C. Berg, Ssd, Single shot multibox detector, in: Computer Vision–ECCV 2016: 14th European Conference, Amsterdam, the Netherlands, October 11–14, 2016, Proceedings, Part 14, Springer, 2016, pp. 21–37.
- [25] L. Ma, L.Q. Zhou, L.Y. Zhao, Y.H. Hu, Classification and recognition of tomato leaf diseases based on deep learning, *J. Chin. Agricult. Mech.* 44 (7) (2023) 187.
- [26] N. Ma, X. Zhang, H.T. Zheng, J. Sun, Shufflenet v2: practical guidelines for efficient cnn architecture design, in: Proceedings of the European Conference on Computer Vision (ECCV), 2018, pp. 116–131.
- [27] J. Redmon, S. Divvala, R. Girshick, A. Farhadi, You only look once: unified, real-time object detection, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 779–788.
- [28] S. Ren, K. He, R. Girshick, J. Sun, Faster r-cnn: towards real-time object detection with region proposal networks, *Adv. Neural Inf. Process. Syst.* 28 (2015).
- [29] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, Liang Chieh Chen, Inverted Residuals and Linear Bottlenecks: Mobile Networks for Classification, Detection and Segmentation, 2018.
- [30] K. Simonyan, A. Zisserman, Very deep convolutional networks for large-scale image recognition, preprint, arXiv:1409.1556, 2014.
- [31] A.I. Studio, Plantvillage dataset (apple insect pest), <https://aistudio.baidu.com/datasetdetail/129485>, 2022.
- [32] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, Andrew Rabinovich, Going deeper with convolutions, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2015, pp. 1–9.
- [33] M. Tan, R. Pang, Q.V. Le, Efficientdet, Scalable and efficient object detection, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2020, pp. 10781–10790.
- [34] Mingxing Tan, Quoc Le, Efficientnet: rethinking model scaling for convolutional neural networks, in: International Conference on Machine Learning, PMLR, 2019, pp. 6105–6114.
- [35] R.J. Wang, X. Li, C.X. Ling, Pelee: a real-time object detection system on mobile devices, *Adv. Neural Inf. Process. Syst.* 31 (2018).
- [36] Z.Q. Wang, X.Y. Yu, X.J. Yang, Y.B. Lan, X.N. Jin, J.Y. Ma, Tomato leaf disease recognition based on wgan and mca-mobilenet, *Trans. Chin. Soc. Agricult. Mach.* (2023).
- [37] Y. Xiong, H. Liu, S. Gupta, B. Akin, G. Bender, Y. Wang, P.J. Kindermans, M. Tan, V. Singh, B. Chen, Mobiledets: searching for object detection architectures for mobile accelerators, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), June 2021, pp. 3825–3834.
- [38] B. Yang, J.P. He, L.N. Zhang, Identification and detection of rice leaf diseases by yolov5 neural network based on improved spp-x, *J. Chin. Agricult. Mech.* (2023).

- [39] R. Ye, B. Li, B. Yan, Z. Li, Lightseg: a light-weight network for real-time semantic segmentation, in: International Conference on Digital Image Processing, 2021.
- [40] D.Q. Zhang, H.Y. Fan, B.S. Kang, J. Gao, T.J. Li, Glacier identification from remote sensing image with shadows using an improved u-net convolutional network, *J. Basic Sci. Eng.* 30 (4) (2022) 13.
- [41] S. Zhang, Y. Xie, J. Wan, H. Xia, S.Z. Li, G. Guo, Widerperson: a diverse dataset for dense pedestrian detection in the wild, *IEEE Trans. Multimed.* 22 (2) (2019) 380–393.
- [42] Xiangyu Zhang, Xinyu Zhou, Mengxiao Lin, Jian Sun, Shufflenet: an extremely efficient convolutional neural network for mobile devices, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2018, pp. 6848–6856.
- [43] F. Zhou, F. He, C. Gui, Z. Dong, M. Xing, Sar target detection based on improved ssd with saliency map and residual network, *Remote Sens.* 14 (1) (2022) 180.